2.1 Konfiguracja edytora Cursor: Indeksowanie kodu, funkcja Composer i praca w trybie terminala AI

Moduł 2: Ekosystem Vibe Coding i AI-first IDE

Krzysztof Rytlewski · 2026

Cursor – czym jest AI-first IDE?

Cursor to edytor kodu zbudowany jako fork VS Code, przeprojektowany od podstaw z myślą o integracji z modelami językowymi. W odróżnieniu od wtyczek, model AI jest tu obywatelem pierwszej klasy – ma dostęp do całego projektu, może modyfikować wiele plików jednocześnie i iteracyjnie poprawiać własne odpowiedzi.

- **AI-first IDE** – interfejs i architektura zoptymalizowane wokół współpracy z modelem językowym.

- **Underlying model** – domyślnie Claude 3.5/3.7 Sonnet lub GPT-4o (konfigurowalne w ustawieniach).

- **Context window** – Cursor wysyła fragmenty kodu jako kontekst; świadome zarządzanie nim jest kluczem do precyzyjnych wyników.

Indeksowanie kodu – jak Cursor „czyta" projekt

Po otwarciu folderu Cursor tworzy lokalny indeks projektu – przekształca kod źródłowy w reprezentacje wektorowe (embeddingi), umożliwiając semantyczne wyszukiwanie po całej bazie kodu.

Jak działa indeksowanie:

1. Cursor skanuje pliki projektu (respektując `.gitignore` i `.cursorignore`).
2. Każdy plik jest dzielony na chunki i zamieniany na wektory.
3. Indeks jest przechowywany lokalnie – nie wysyłany na serwery bez zgody (tryb Privacy Mode).
4. Przy pytaniu w Chat Cursor wykonuje similarity search i dołącza najtrafniejsze fragmenty do modelu.

Plik `.cursorignore` – kontrola indeksu

Plik `.cursorignore` działa jak `.gitignore` – wyklucza foldery i pliki, które nie powinny trafić do indeksu. Dobra konfiguracja **przyspiesza indeksowanie** i **poprawia jakość odpowiedzi**.

```
# Przykład .cursorignore dla projektu Node.js + Angular
node_modules/
dist/
.angular/
coverage/
*.log
.env
```

Ręczne odświeżenie indeksu: `Ctrl+Shift+P` → *Cursor: Reindex Codebase*.

Im lepiej opisany kod (nazwy funkcji, komentarze), tym trafniej Cursor lokalizuje odpowiednie fragmenty.

Chat i Inline Edit – podstawy interakcji

Cursor oferuje dwa główne tryby pracy z AI:

Chat (`Ctrl+L`) – panel boczny do rozmowy z modelem w kontekście całego projektu:

- `@codebase` – przeszuka indeks semantycznie,
- `@filename` – dołącza konkretny plik do kontekstu,
- wygenerowany kod można zaakceptować jednym kliknięciem.

Inline Edit (`Ctrl+K`) – mini-okno bezpośrednio w edytorze. Zaznaczasz fragment kodu, opisujesz zmianę, model pokazuje diff do zaakceptowania lub odrzucenia.

- Chat: eksploracja i planowanie.
- Inline Edit: precyzyjne, lokalne modyfikacje.

Inline Edit – przykład refaktoryzacji

Zastosowanie `Ctrl+K` z promptem: *„rename parameters, add JSDoc, return early if no roles"*:

```
// Przed: nieczytelna funkcja
function chk(u, r) { return u.roles && u.roles.includes(r); }

// Po: po Inline Edit
/**
 * Sprawdza, czy użytkownik posiada wskazaną rolę.
 * @param {Object} user - Obiekt użytkownika
 * @param {string} role - Nazwa roli do sprawdzenia
 * @returns {boolean}
 */
function hasRole(user, role) {
  if (!user.roles) return false;
  return user.roles.includes(role);
}
```

Composer – praca z wieloma plikami jednocześnie

Composer (`Ctrl+I`) to najbardziej zaawansowana funkcja Cursora. Umożliwia **edycję wielu plików w ramach jednego zapytania** – model tworzy nowe pliki, modyfikuje istniejące i wyświetla kompletny diff do zatwierdzenia.

Kiedy używać Composera:

- Dodawanie funkcjonalności obejmującej kilka warstw (np. endpoint + serwis + moduł Angular).
- Refaktoryzacja całego modułu (np. migracja z `Promise` na `async/await`).
- Generowanie scaffoldingu (struktura katalogów, boilerplate).
- Jednoczesna aktualizacja testów i implementacji.

Composer wyświetla listę planowanych zmian przed zapisaniem – możesz akceptować lub odrzucać poszczególne pliki.

Composer – przykład prompta

Przykład precyzyjnego prompta do Composera:

> *Stwórz endpoint* `POST /api/users/:id/avatar` *w Express.js. Endpoint powinien:*
>
> - przyjmować plik multipart/form-data,
> - walidować typ MIME (tylko `image/jpeg`, `image/png`),
> - zapisywać plik do folderu `uploads/avatars/`,
> - zwracać ścieżkę do pliku w odpowiedzi JSON. Zaktualizuj też plik routera i dodaj test jednostkowy w Jest.

Tryb Agent w Composerze pozwala modelowi na wykonywanie poleceń w terminalu i iteracyjne poprawianie kodu na podstawie błędów kompilacji lub testów – bez interwencji użytkownika między krokami.

Praca w trybie terminala AI

`Ctrl+K` w terminalu – zamiast wpisywać polecenie ręcznie, opisujesz cel w języku naturalnym, a Cursor generuje komendę do zatwierdzenia.

```
# Opis: "znajdź wszystkie pliki TypeScript zmodyfikowane w ciągu ostatnich 7 dni"
find . -name "*.ts" -newer $(date -d '7 days ago' +%Y-%m-%d) \
  -not -path "*/node_modules/*"

# Opis: "uruchom testy tylko dla modułu 'auth', z flagą verbose"
npx jest --testPathPattern="auth" --verbose
```

Terminal Debug – gdy pojawia się błąd, kliknij *Debug with AI* (`Ctrl+Shift+R`), aby Cursor przeanalizował traceback i zaproponował poprawkę bezpośrednio w kodzie.

Plik `.cursorrules` – stały kontekst projektu

`.cursorrules` to mechanizm dostarczania **stałego systemu promptu na poziomie projektu**. Instrukcje są dołączane do każdego zapytania.

```
## Stack
- Backend: Node.js 20 + Express + MongoDB (Mongoose)
- Frontend: Angular 17 (standalone components, signals)
- Testy: Jest (backend), Karma + Jasmine (frontend)

## Konwencje
- Nazwy plików: kebab-case
- Klasy i interfejsy: PascalCase
- Zawsze typuj odpowiedzi API (interface + DTO)

## Ważne
- Nie używaj `var`, tylko `const` i `let`
- Asynchroniczność: wyłącznie async/await
```

Dobrze skonfigurowany `.cursorrules` redukuje konieczność powtarzania instrukcji i poprawia spójność generowanego kodu.

Rekomendowany workflow dla nowej funkcjonalności

Spójny workflow minimalizuje context-switching i maksymalizuje precyzję modelu:

1. **Planowanie w Chat** (`Ctrl+L` z `@codebase`) – identyfikuj pliki i potencjalne problemy.
2. **Implementacja w Composerze** (`Ctrl+I`) – przekaż plan, wskaż pliki kontekstu `@plik1 @plik2`.
3. **Przegląd i akceptacja** – przejrzyj każdy plik, nie akceptuj wszystkiego na ślepo.
4. **Korekty Inline** (`Ctrl+K`) – dla drobnych poprawek po przeglądzie.
5. **Testy w terminalu** (`Ctrl+K` w terminalu) – uruchom testy i użyj *Debug with AI* przy błędach.

Typowe pułapki przy pracy z Cursorem

Świadomość pułapek pozwala unikać strat czasu i błędów logicznych w generowanym kodzie:

- **Zbyt duże prompty** – Composer traci fokus przy zbyt wielu wymaganiach naraz. Dziel zadania na etapy.

- **Brak kontekstu** – bez `@pliku` lub `@codebase` model generuje kod „w próżni", niekompatybilny z projektem.

- **Ślepa akceptacja diffów** – zawsze czytaj wygenerowany kod. Modele popełniają błędy logiczne, których nie wychwytuje kompilator.